



第三章 程序的转换及机器级表示

主要内容



华中科技大学

- ◆ 3.1 程序转换概述
- ◆ 3.2 IA-32指令系统概述
- ◆ 3.3 IA-32常用指令类型及其操作
- ◆ 3.4 C语言的机器级表示
- ◆ 3.5 复杂数据类型的分配和访问
- ◆ 3.6 越界访问和缓冲区攻击





一、数据类型及其格式

IA-32指令系统中，操作数可以按存储方式分为：

- 立即数
- 寄存器操作数
- 存储器操作数

注：IA-32指令系统中，操作数被看作0/1序列。





AT&T格式指令格式

AT&T格式和Intel格式的汇编指令对比：

Intel格式	AT&T格式
MOV EAX, 0	movl \$0, %eax
ADD EAX, EBX	addl %ebx, %eax
INC ECX	incl %ecx

注：两者目的操作数和源操作数的顺序不同





AT&T指令长度后缀

AT&T格式汇编指令，可以有操作数长度后缀：

后缀	操作数	例子
b	8位	<code>movb \$0, %al,</code>
w	16位	<code>movw \$0, buf</code>
l	32位	<code>movl \$0, %eax</code>

注：若指令中有一个操作数是寄存器，则操作数长度已确定，长度后缀可省略





二、寄存器组织

	31	16 15	8 7	0	
EAX			AH (AX) AL		累加器
EBX			BH (BX) BL		基址寄存器
ECX			CH (CX) CL		计数寄存器
EDX			DH (DX) DL		数据寄存器
ESP			SP		栈指针寄存器
EBP			BP		基址指针寄存器
ESI			SI		源变址寄存器
EDI			DI		目标变址寄存器
EIP			IP		指令指针寄存器
EFLAGS			FLAGS		标志寄存器
			CS		代码段
			SS		堆栈段
			DS		数据段
			ES		附加段
			FS		附加段
			GS		附加段

图 3.2 IA-32 的定点寄存器组

三、寻址方式



华中科技大学

1.主存储器

	主存	辅存
容量	小	大
存取速度	快	慢
价格	高	低





1. 主存储器

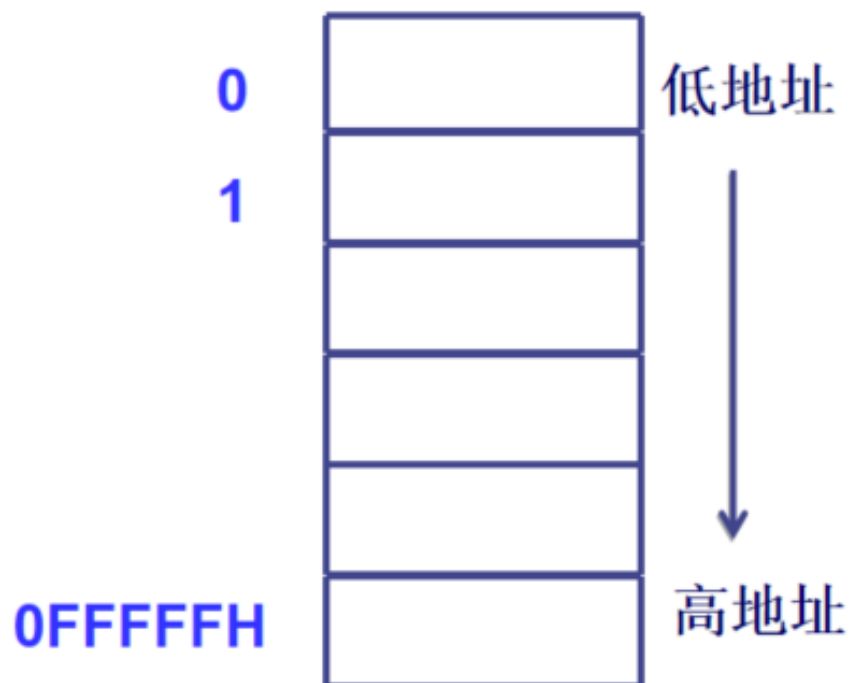
- 主存储器的基本存储单位是“位” (bit)
- 每8位组合成一个“字节”
- 每相邻的两个字节称为“字”
- 相邻的四个字节称为“双字”
- 为了区别各个存储单元，给每个单元编号，称为地址。

地址



华中科技大学

字节单元的编号，也称为物理地址

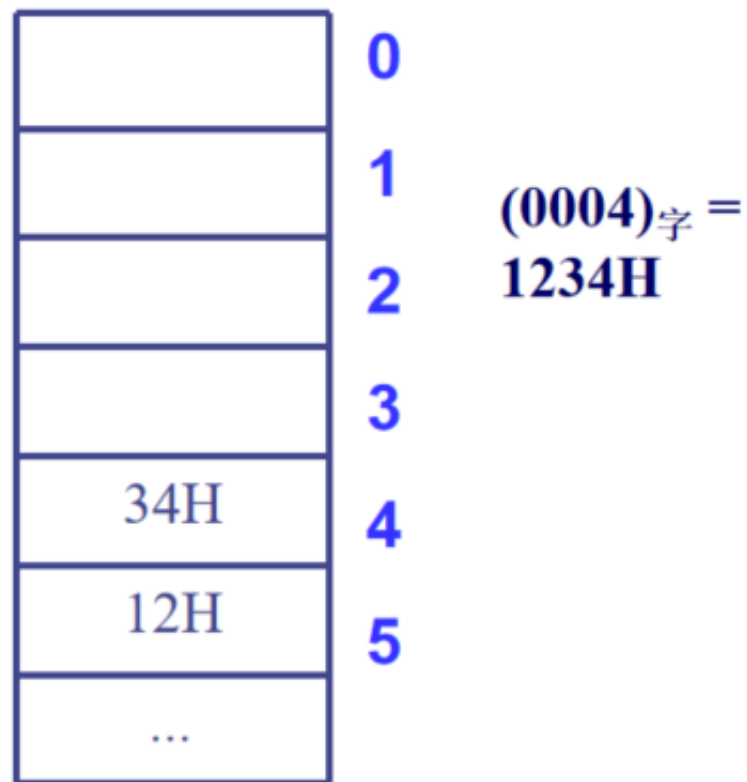


大小端



华中科技大学

低字节放低位，高字节放高位





2. 存储器物理地址的形成

早期的8086微处理:

- ◆ 20位地址总线, 寻址范围: $2^{20} = 1\text{M}$
- ◆ 与地址相关的寄存器均为16位(SP, BP, SI, DI),
寻址范围: $2^{16} = 64\text{K}$

问题: 如何通过16位寄存器访问1MB的内存

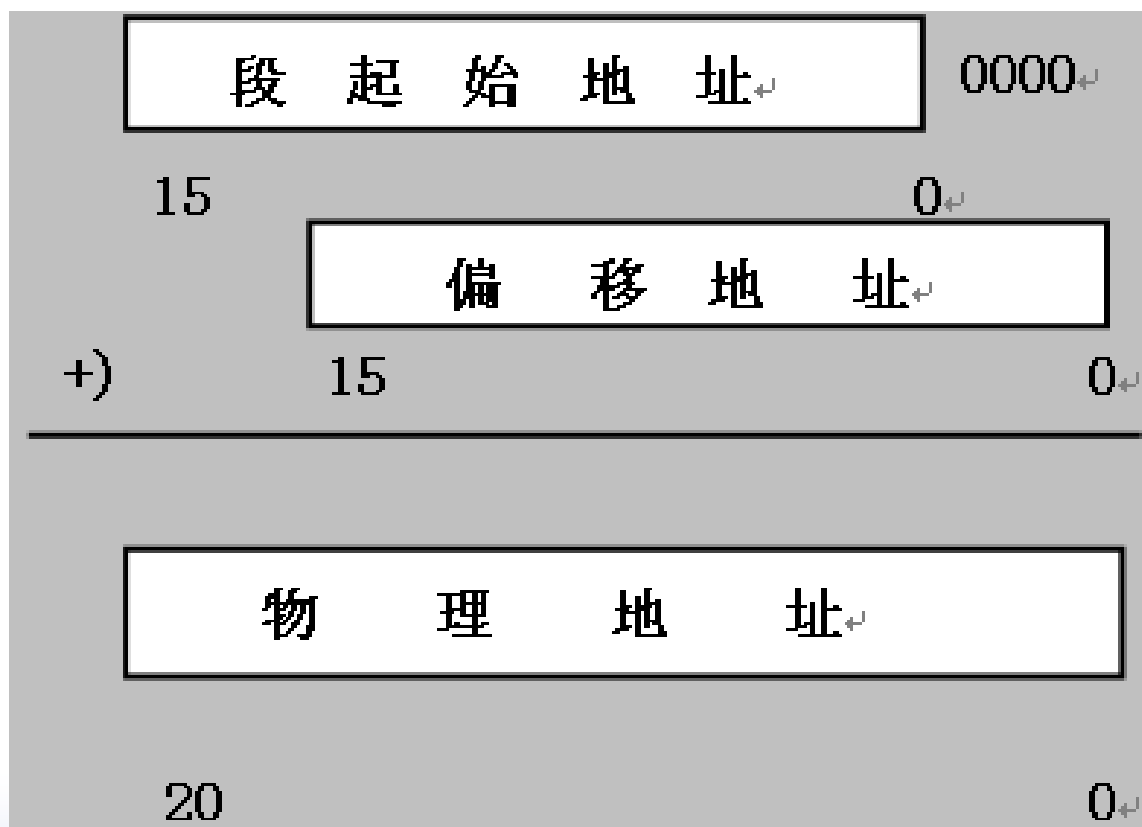
解决: 将1M字节主存分段, 每段最多64K字节

- ◆ 用CS、DS、SS、ES保存当前可用段的段首地址
- ◆ 计算物理地址时, 应将段寄存器内容左移4位, 然后再与偏移地址相加, 得到待访问单元的物理地址。



2. 存储器物理地址的形成

$$PA = \text{段首址} * 16 + \text{偏移地址}$$





3. 保护模式下的内存管理

80x86的寻址范围:

- ◆ 80x86地址总线32位, 寻址范围4G
- ◆ 80x86通用寄存器32位, 寻址范围也是4G

所以80x86的一个通用寄存器能够存放一个完整的
32位地址



3. 保护模式下的内存管理

要为每个进程提供一个独立的4G大小的虚拟地址空间。
逻辑地址空间（多个进程的地址空间总和）大于物理地址空间。采用**分页机制**来实现：

- ◆ 将物理地址映射到进程的逻辑地址空间上；
- ◆ 只有映射了物理内存的逻辑地址空间才能访问；
- ◆ 每次映射的物理内存不大，且用完后可释放，再重新映射到新的逻辑地址空间上，因此能够让多个进程同时运行；
- ◆ 分页由系统基于页目录和页表，自动完成





3.保护模式下的内存管理

进程1的逻辑地址空间



物理内存



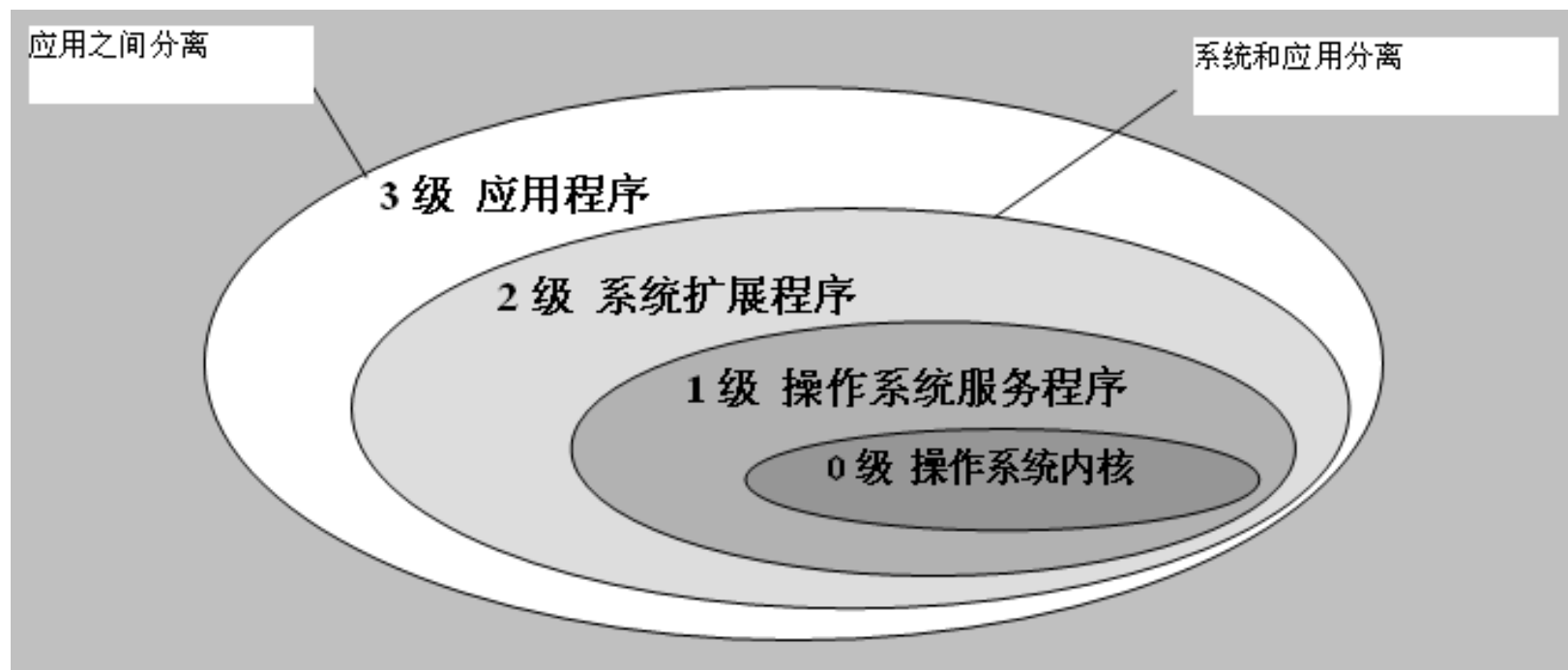
进程2的逻辑地址空间





特权级与分段机制

(1) 80x86有**4个特权级**: 0,1,2,3 (0级最高, 3级最低), 能够在虚拟内存的基础上实现进一步的**内存保护机制**

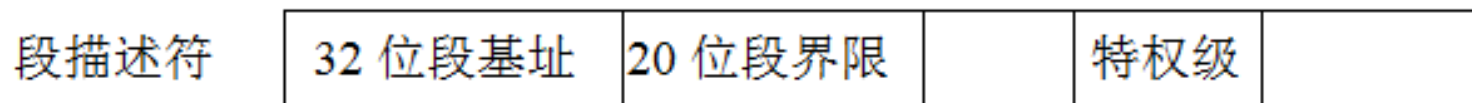
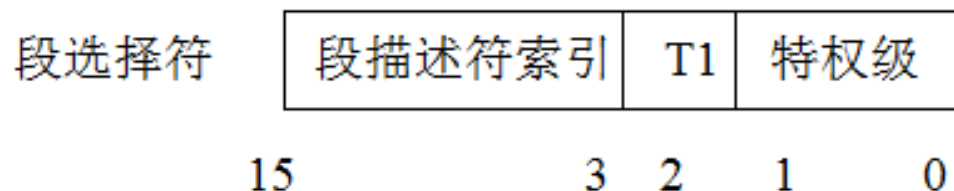




特权级与分段机制

(2) 80x86通过**分段机制**来实现特权级的访问控制。

用“**段选择符**: EA”的形式表示逻辑地址，段选择符（16位）存放在段寄存器中，指向一个**段描述符**（64位）



用逻辑地址访问内存时，段选择符中的特权级需要高于段描述符中的**特权级**。





4.具体寻址方式

寻址方法

立即寻址方式

寄存器寻址方式

直接寻址方式

寄存器间接寻址方式

变址寻址方式

基址加变址寻址方式



4.具体寻址方式

(1)立即寻址

操作数直接存放在指令中，紧跟在操作码之后，它作为指令的一部分存放在代码段里，这种操作数称为立即数。

汇编格式：\$n

功能：指令的下一单元的内容为操作数n，即：





(1)立即寻址

例: `movl $0x3064, %eax`

目的操作数采用寄存器寻址，为寄存器eax。

源操作数采用立即寻址。即：

MOV
64H
30H
00H
00H



(1)立即寻址

说明:

- ◆ 立即数只能作为双操作数指令的源操作数，不能作为目的操作数；

如: `movl %ax, $100` ; error

- ◆ 立即数不能作为单操作数指令的操作数；

如: `incl $50` ; error

- ◆ 立即数只有大小，没有类型，未分配内存单元；

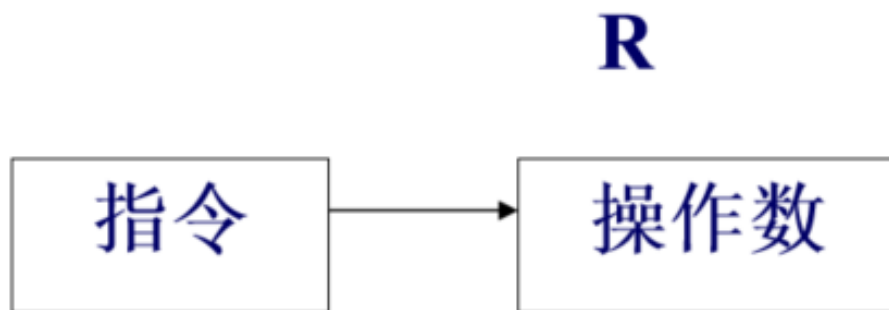


(2) 寄存器寻址

在这种寻址方式中，指令所指明的寄存器中存放操作数

汇编格式：%R

功能：寄存器R的内容就是操作数。





(2) 寄存器寻址

例: `incl %eax`

操作数在EAX中。



执行前: $EAX = 12H$

执行: $EAX + 1 \rightarrow EAX$ 。

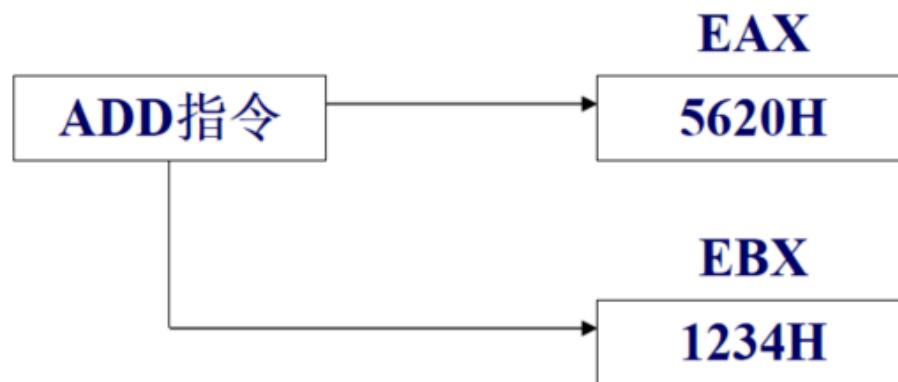
执行后: $EAX = 13H$



(2) 寄存器寻址

例: `add %ebx, %eax`

EAX为目的操作数地址, EBX为源操作数地址。



执行前: `EAX=1234H, EBX=5620H`

执行: `EAX+EBX→EAX。`

执行后: `EAX=6854H, EBX=5620H`

目的操作数、源操作数都是用寄存器寻址



(2) 寄存器寻址

说明:

- ◆ 采用寄存器寻址方式，指令执行速度快。特别是加法指令，如果目的操作数为AX，指令执行快一些。
- ◆ 采用寄存器寻址方式，目的、源操作数类型必须一致。如：movw %bx, %ah; ERROR

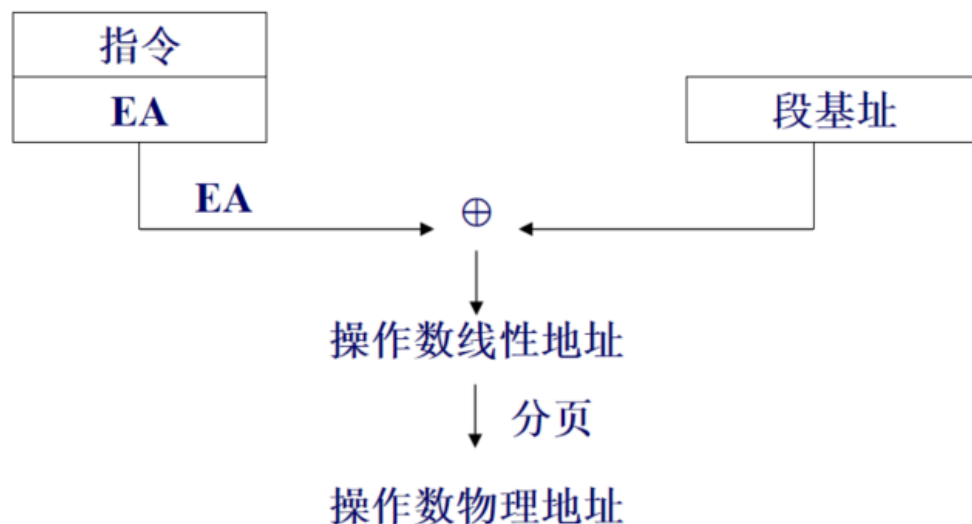
(3) 直接寻址

直接寻址方式中，操作数的偏移地址就在指令操作码后面，而操作数则存放在内存中。

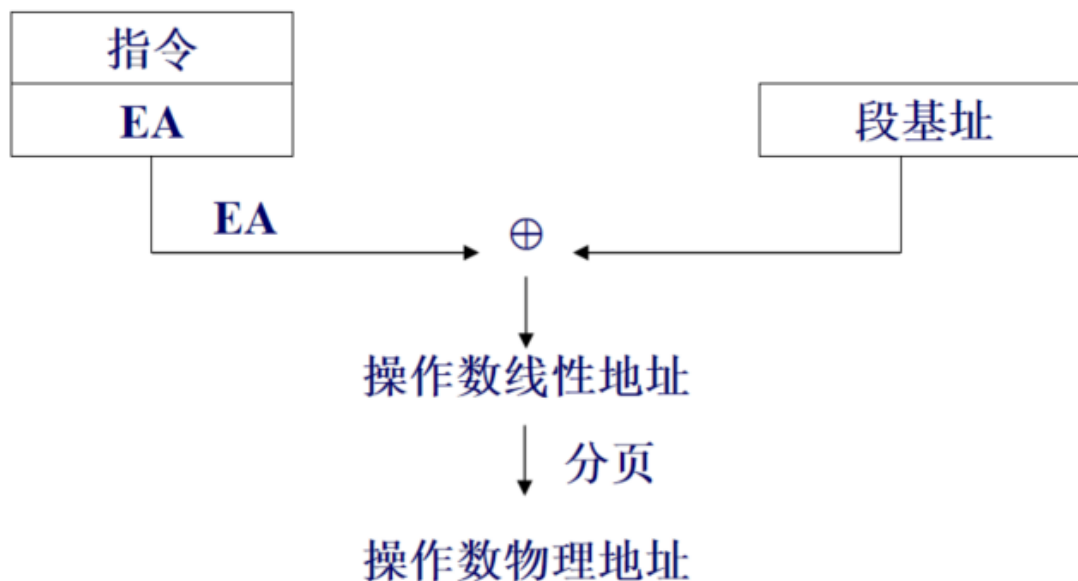
汇编格式：EA

或：地址表达式

功能：指令下一字单元的内容是操作数的偏移地址。



(3) 直接寻址



段基址为0时，EA等于操作数的线性地址。

操作数的线性地址到物理地址的转换，由系统自动完成。

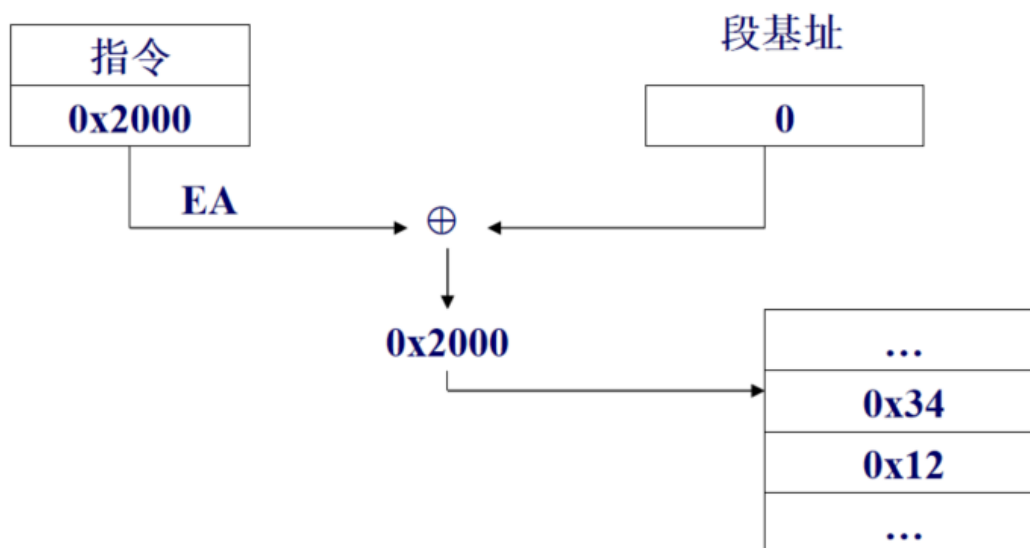
因此在实际应用中，只需要着重考虑操作数的偏移地址。



(3)直接寻址

例: **movl 0x2000, %eax**

执行前: **(0x2000) = 1234H**



执行: **(2000H) $\rightarrow$ EAX**

执行后: **EAX = 0x1234**





(3) 直接寻址

说明:

- ◆ “EA” 格式表示的操作数无类型。
- ◆ “地址表达式” 格式中，若还有变量，则表示的操作数有类型。

例:

`movw 0x20, %ax` ; 源操作数无类型

`movw %ax, buf` ; 目的操作数为直接寻

址，其中buf为已定义的字类型的变量。则目的操作数有类型

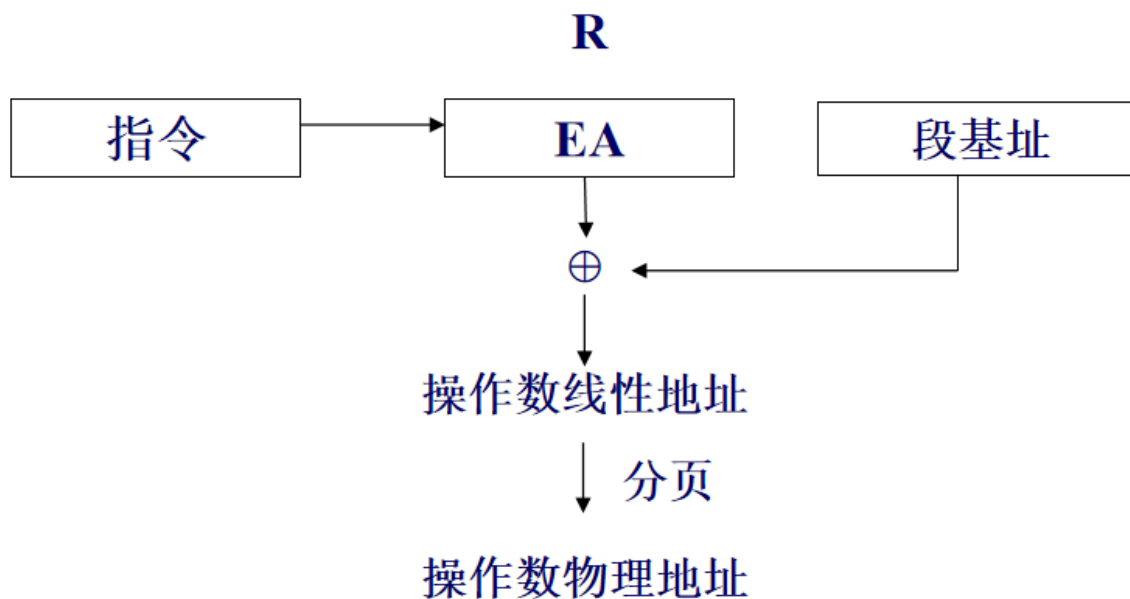


(4) 寄存器间接寻址

操作数的偏移地址在寄存器中，而操作数则在存储器中。

格式：(%R)

功能：R的内容为操作数的偏移地址EA



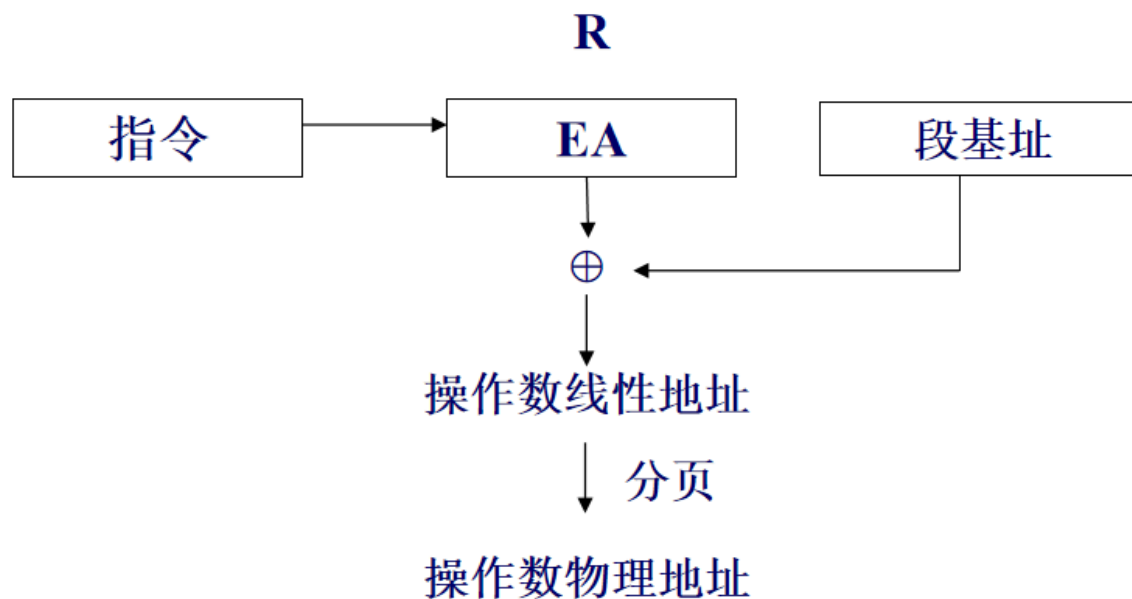


(4) 寄存器间接寻址

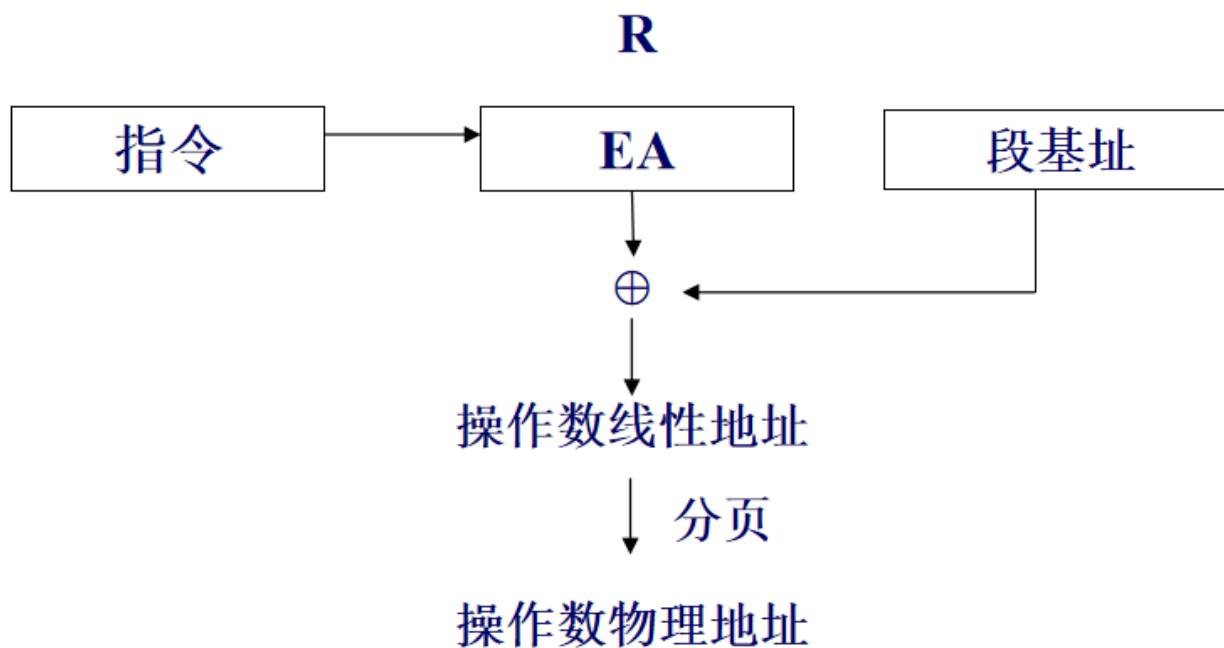
操作数的偏移地址在寄存器中，而操作数则在存储器中。

格式：(**%R**)

功能：R的内容为操作数的偏移地址EA



(4) 寄存器间接寻址



R可以是EAX、EBX、ECX、EDX、ESI、EDI、EBP、ESP之一。EBP和ESP默认所指的段为堆栈段。

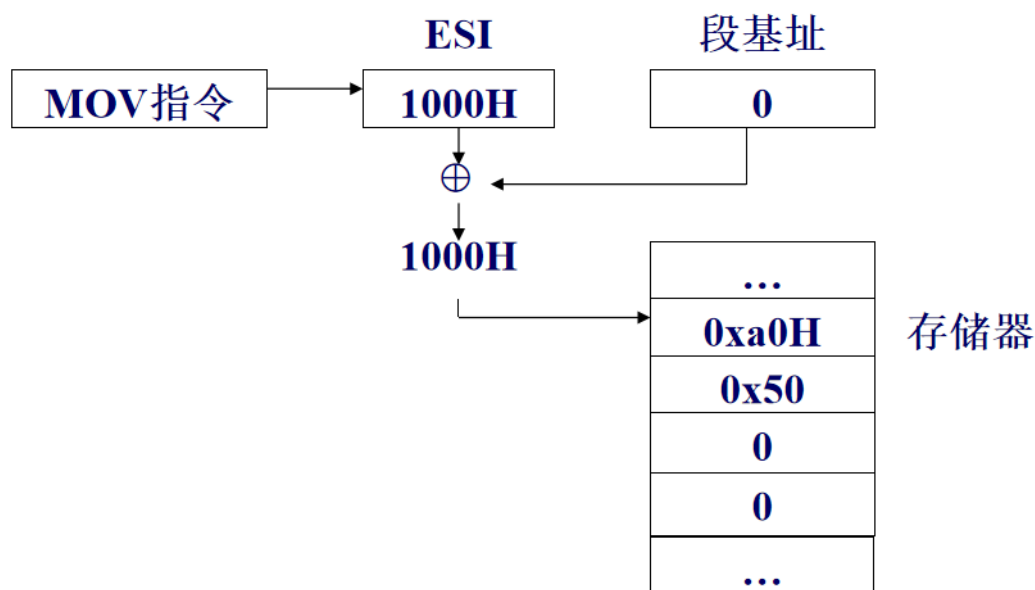


(4) 寄存器间接寻址

例:

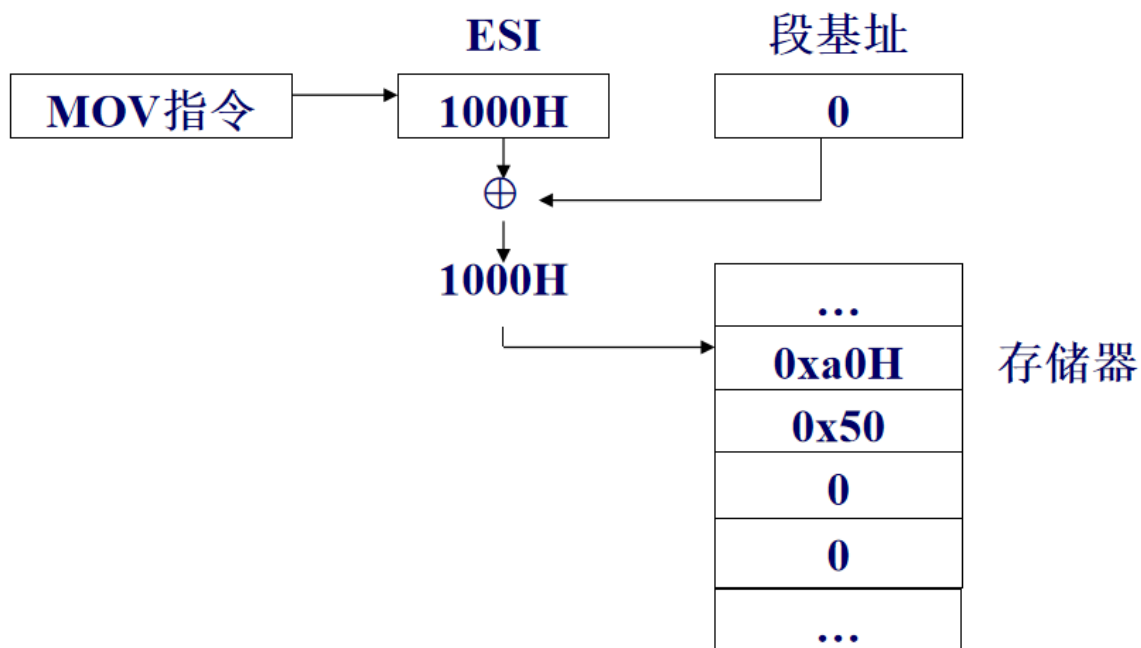
`mov (%esi), %eax`

假定执行前: $EAX = 5$, $ESI = 1000H$, $(1000H) = 50A0H$ 。





(4) 寄存器间接寻址



执行: (1000H) $\rightarrow$ EAX

执行后: EAX=50A0H, ESI、DS、(1000H) 不变。



(4) 寄存器间接寻址

说明:

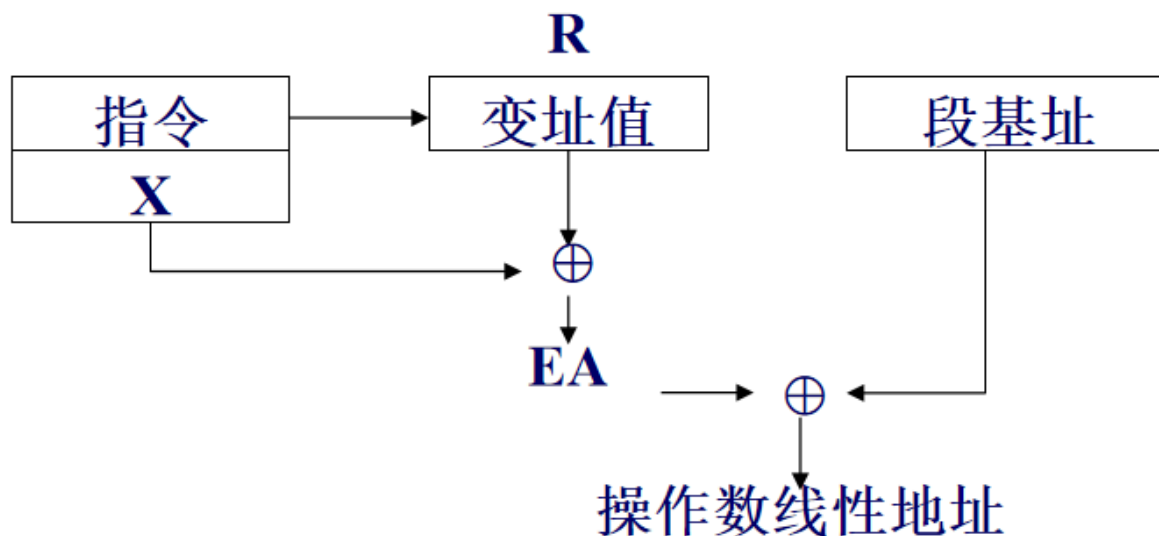
- ◆ 当R是EBP、ESP, 则操作对象在堆栈段中;
当R为其他寄存器, 则操作对象在数据段中。
- ◆ 此种寻址无类型

(5) 变址寻址

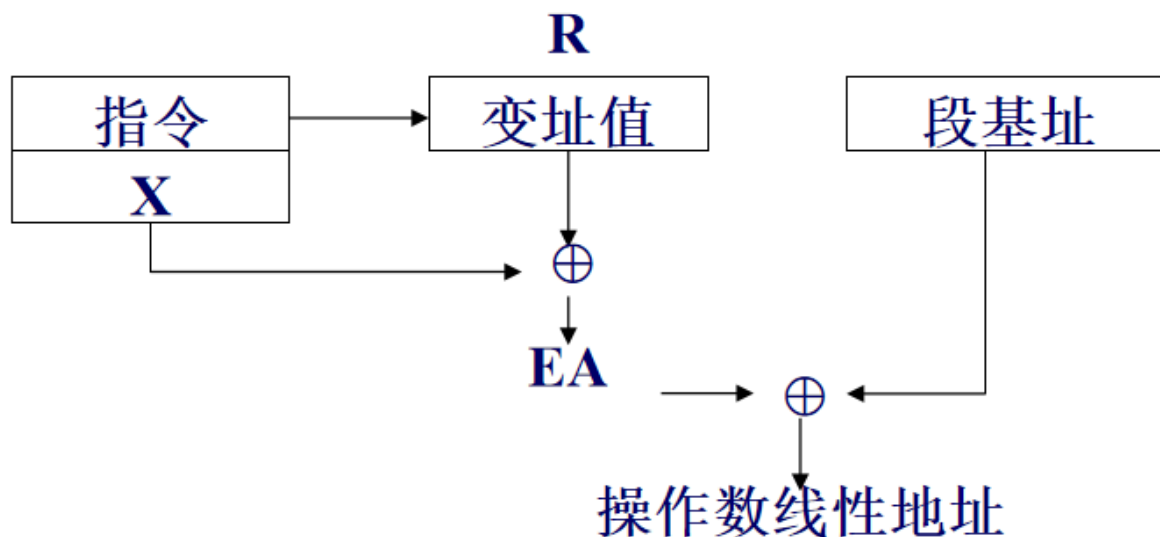
操作数的偏移地址EA是指令中指定的寄存器的内容与指令中给出的位移量之和。操作数在存储器中。

汇编格式：X(%R) (或：X(, %R, S))

功能：R的内容与X相加，和为操作数的偏移地址EA。



(5) 变址寻址



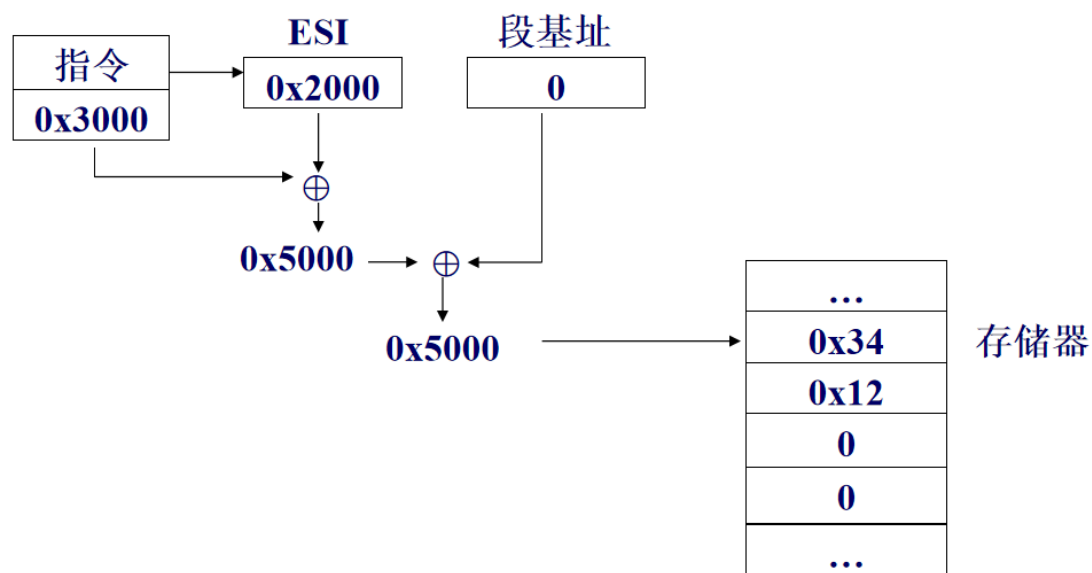
寄存器的选择同寄存器间接寻址一样，可以用EAX、EBX、ECX、EDX、ESI、EDI、EBP、ESP之一。EBP和ESP所指的段默认为堆栈段。

(5) 变址寻址

例: `movl count(%esi), eax`

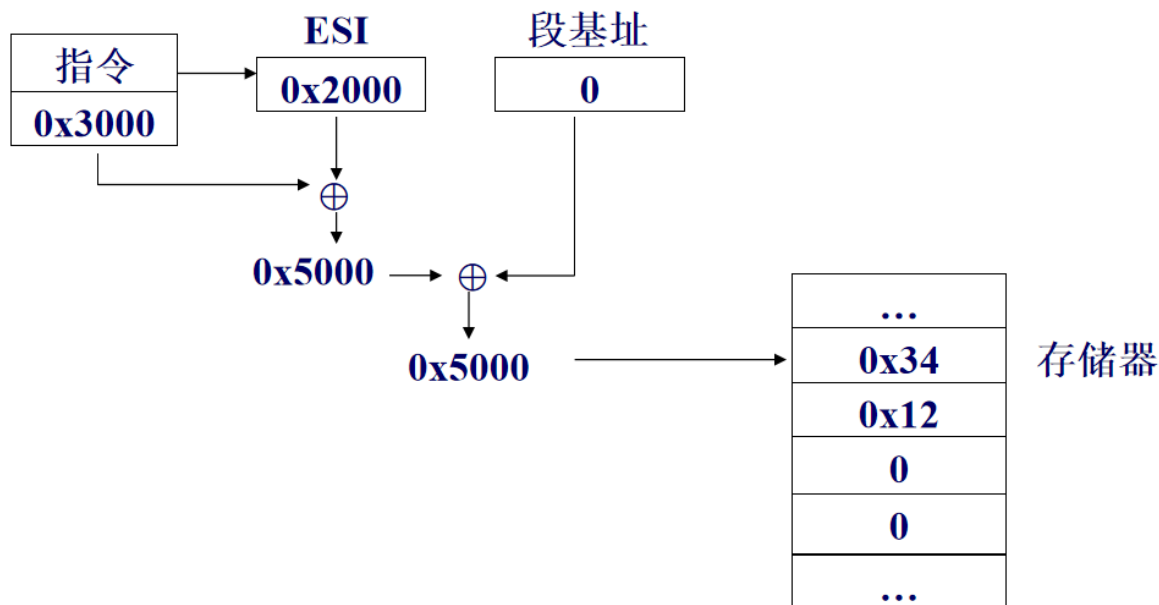
源操作数采用变址寻址方式。操作数EA=ESI + COUNT。

假定执行前: ESI=2000H, COUNT=3000H, EAX=0,
(5000H) =1234H。





(5) 变址寻址



执行: $(0x5000) \rightarrow EAX$

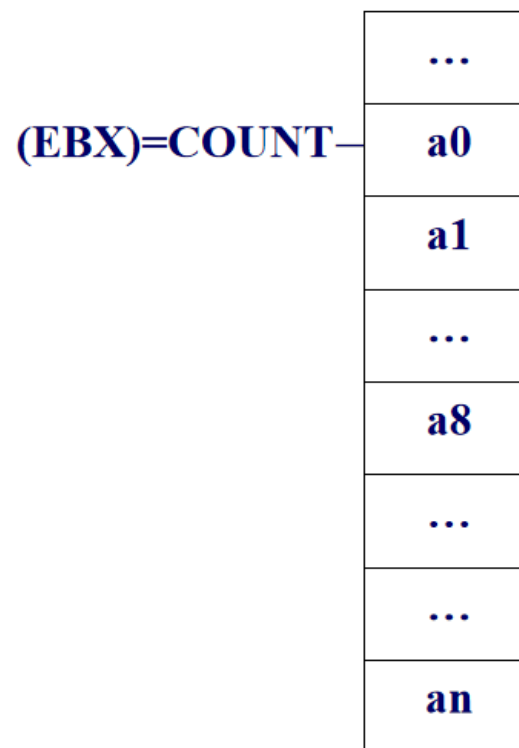
执行后: $EAX = 0x1234$, ESI、DS、 $(5000H)$ 内容不变。



(5) 变址寻址

采用变址寻址可以方便地访问数组中的任一元素，使得编程更加方便。

例：一维数组a，其存放如图所示。





(5) 变址寻址

访问数组元素的源码如下：

；定义数据段：

```
.section .data
```

```
count      .byte a0, a1,..., an
```

```
.section .text
```

...

```
mov  offset count, %ebx
```

；若要将a8放到ah中，可用：

```
movb 8(%ebx), %ah
```

；若要将第 i 个元素放到AH中，可用：

```
movb i(%ebx), %ah
```

...

(EBX)=COUNT

...
a0
a1
...
a8
...
...
an



(5) 变址寻址

说明:

◆ 段属性问题

X为常数或数值表达式时, 操作对象的段由R决定

X为变量或标号时, 操作数对象所在的段就是变量或标号所在的段

。

◆ 类型

当X为常量时, 此种寻址无类型;

当X为含有变量或者标号的表达式时, 则有类型。

如: `movw buf(%ebx), %cx;`

若BUF是已定义的字类型变量, `buf(%ebx)`类型为字类型。

若BUF定义为字节类型, 则`buf(%ebx)`与`%cx`类型不一致。



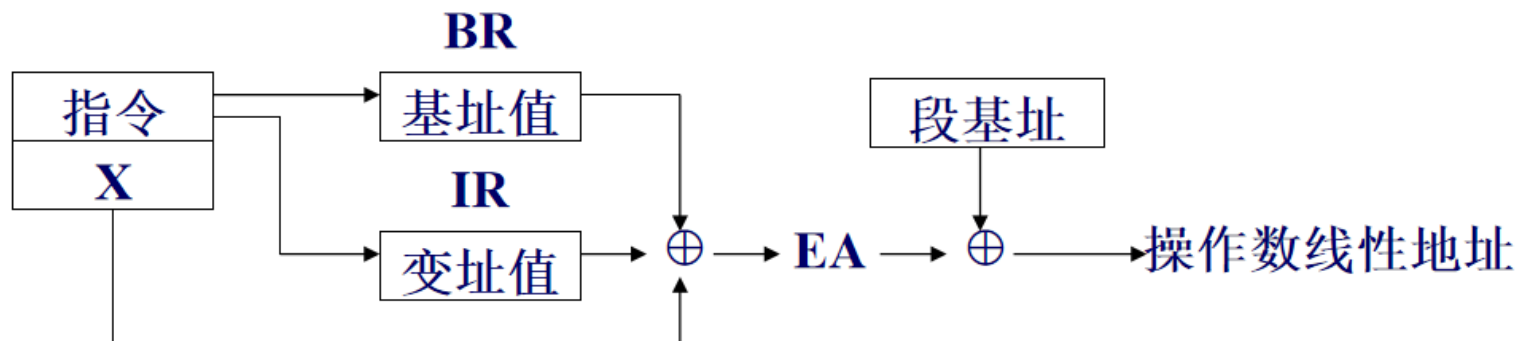
(6) 基址加变址寻址

操作数的偏移地址EA是指令中指定的基址寄存器内容、变址寄存器内容及位移量X三者之和。操作数存放在主存之中。

格式: $X(\%BR, \%IR, S)$

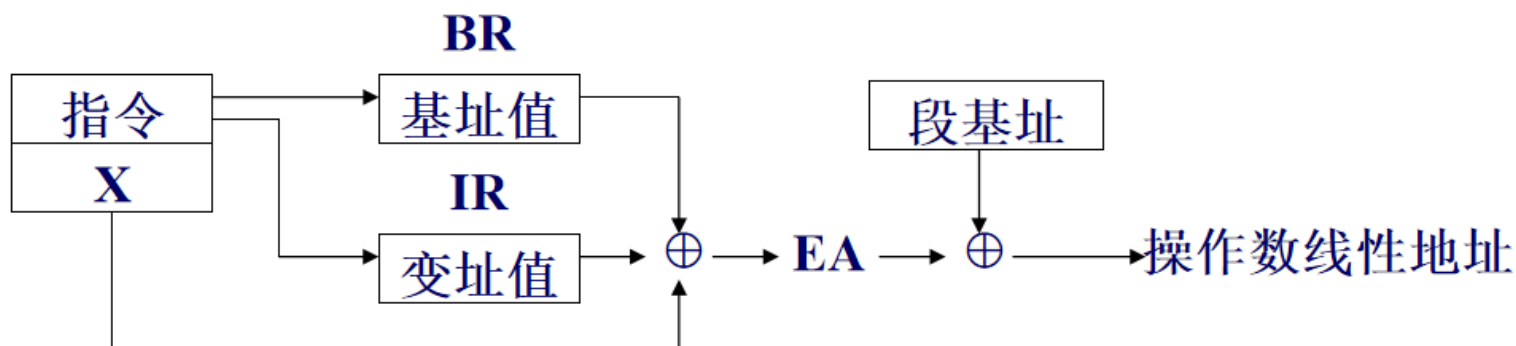
功能: BR的内容加上 $IR * S$, 再加上X, 得到操作数的偏移地址。

也就是: $EA = BR + IR * S + X$ 。





(6) 基址加变址寻址



其中：BR表示基址寄存器，IR表示变址寄存器。BR和IR可以是EAX、EBX、ECX、EDX、ESI、EDI、EBP、ESP之一。

若BR为空，则简化为变址寻址。

基址寄存器决定操作数所在的段。如果选用EBP或ESP，则操作数在堆栈段中；否则操作数在数据段中。

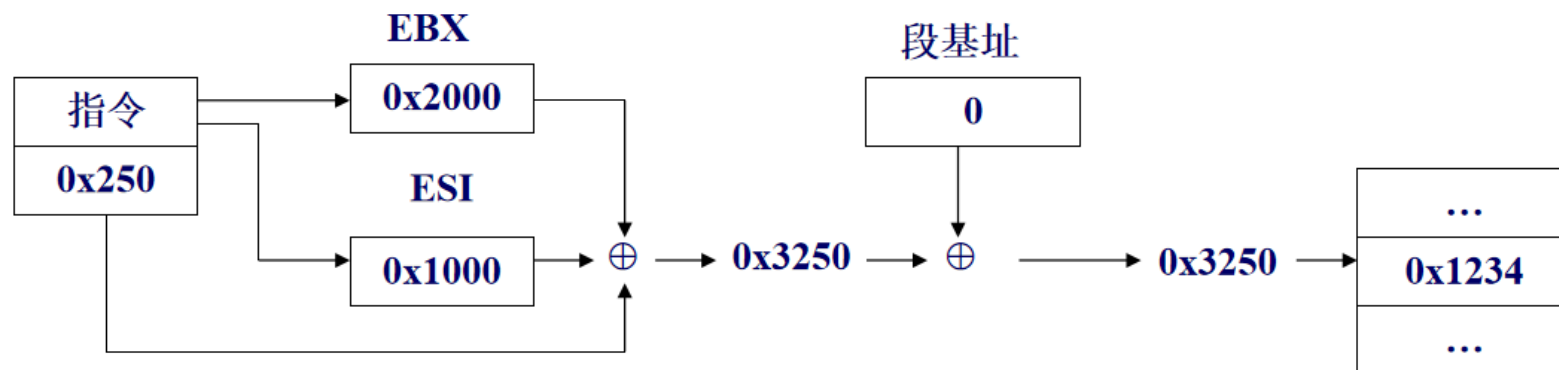
(6) 基址加变址寻址

例: `movw mask(%ebx, %esi, ), %ax`; mask为一个符号常量

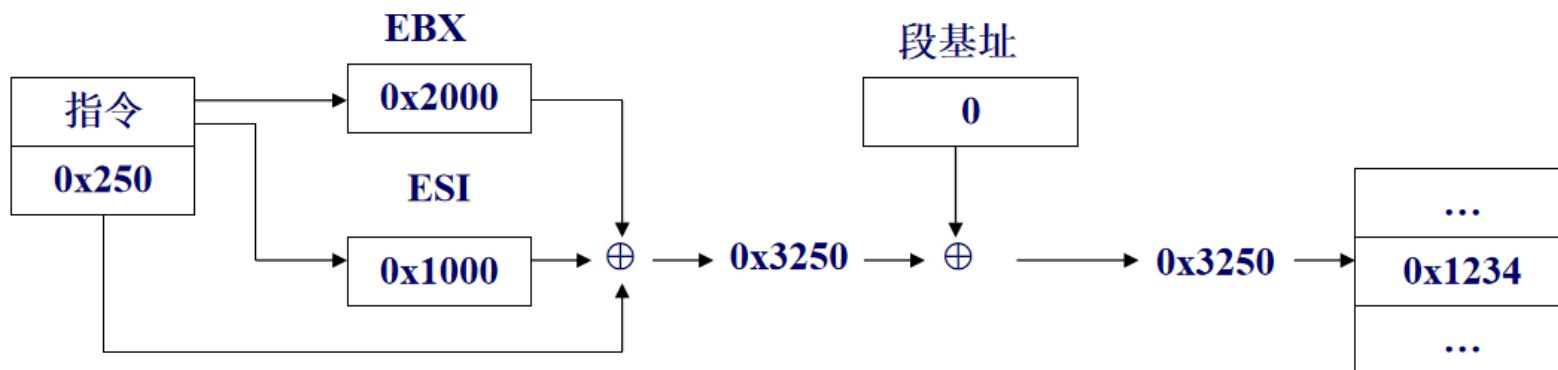
源操作数采用基址加变址寻址, $EA = EBX + ESI + mask$

执行前: $EBX=0x2000$, $ESI=0x1000$, $MASK=0x250$,

($0x2000 + 0x1000 + 0x250 = 0x3250$)



(6) 基址加变址寻址



执行: (3250H) $\rightarrow$ AX

执行后: AX=0x1234, EBX、ESI、DS、(0x3250) 不变。



(6) 基址加变址寻址

说明:

段的默认情况: 当X为常数时, BR为BP默认段为SS, 其它默认段为DS。操作数无类型。

当X为变量或标号表达式时, 操作对象所在的段就是变量或标号所在的段。操作数有类型, 且与变量或标号类型相同。





带有比例因子的寻址方式

$X(\%BR, \%IR, S)$

(1) 只在变址寄存器IR上可以加比例因子S

(2) 比例因子S可以为1, 2, 4, 8

CPU提供的指令和寻址方式，需要能够将高级语言翻译成高效的机器代码。





帶有比例因子的尋址方式

例：

```
movw buf(%ebx, %esi, 2), %ax
```

則 $EA = EBX + 2 * ESI + BUF$ 的偏移地址



(7)寻址方式的有关问题

名称	操作数存放位置	偏移量	地址寄存器
立即数	常数	×	×
寄存器	寄存器	×	×
直接	内存	√	×
间接	内存	×	√
变址	内存	√	√
基址加变址	内存	√	两个



(7)寻址方式的有关问题

- ◆ 某些指令的执行要涉及到**缺省的操作数**，其寻址方式也是隐含的。

例： `cbw`

隐含源和目的操作数。源操作数为AL,寄存器寻址；
目的操作数为AX,寄存器寻址。

- ◆ 双操作数指令中，源和目的操作数**不能同为存储器**
寻址方式

例： `movb (%esi), (%edi)        ; ERROR`

`addl 10(%ebx, %edi, 4), count        ;ERROR`





(8) 寻址方式举例

例：已知数据段定义和存储示意图如下：

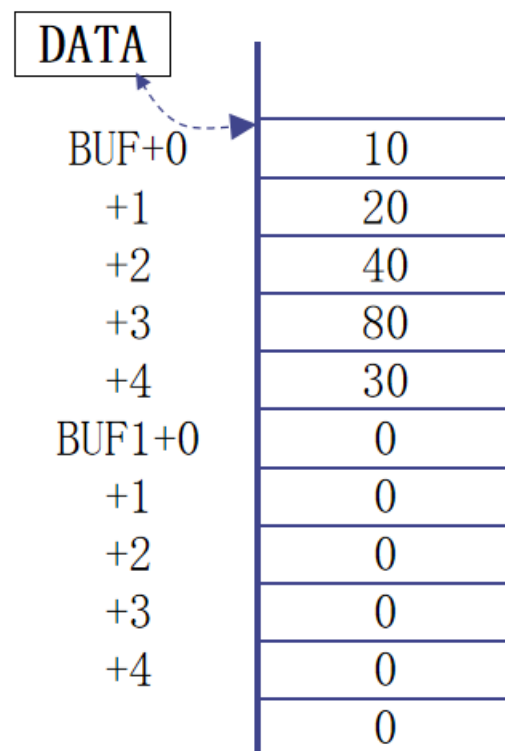
#

```
.section .data
```

```
buf: .byte 10, 20, 40, 80, 30
```

```
buf1: .byte 0, 0, 0, 0, 0
```

1) 分别利用直接寻址，寄存器间接寻址，变址寻址和基址加变址寻址，将该段中BUF+3单元中的内容送到AL中。





(8) 寻址方式举例

1) 分别利用直接寻址, 寄存器间接寻址, 变址寻址和基址加变址寻址, 将该段中BUF+3单元中的内容送到AL中。

a) 直接寻址

```
mov buf(3), %al
```

b) 寄存器间接寻址

```
lea buf(3), %ebx ;将  
                BUF+3相对于段首  
地    址的偏移地址EA=3  
        送入BX中
```

```
mov (%ebx), %al
```

c) 寄存器变址寻址

```
leal buf, %ebx  
movb 3(%ebx), %al
```

d) 基址加变址寻址

```
leal buf, %ebx  
movl $3, %esi  
movb (%ebx, %esi, ), %al
```





(8) 寻址方式举例

2) 利用寄存器间接寻址, 变址寻址和基址加变址寻址, 将BUF为首址的连续字节单元的内容分别送到以BUF1为首址的连续字节单元中。

a) 寄存器间接寻址

```
...  
leal buf, %esi  
leal buf1, %edi  
movl $5, %ecx  
l1:  
movb (%esi), %al  
movb %al, (%edi)  
inc %esi  
inc %edi  
dec %ecx  
jnz l1  
...
```

b) 变址寻址

```
...  
movl $0, %esi  
movl $5, %ecx  
l1:  
movb buf(%esi), %al  
movb %al, buf1(%esi)  
inc %esi  
dec %ecx  
jnz l1  
...
```

c) 基址加变址寻址

```
...  
leal buf, %ebx  
leal buf1, %ebp  
movl $0, %esi  
movl $5, %ecx  
l1:  
movb (%ebx, %esi, 1), %al  
movb %al, (%ebp, %esi, 1)  
inc %esi  
dec %ecx  
jnz l1  
...
```



(8) 寻址方式举例

2) 利用寄存器间接寻址，变址寻址和基址加变址寻址，将BUF为首址的连续字节单元的内容分别送到以BUF1为首址的连续字节单元中。

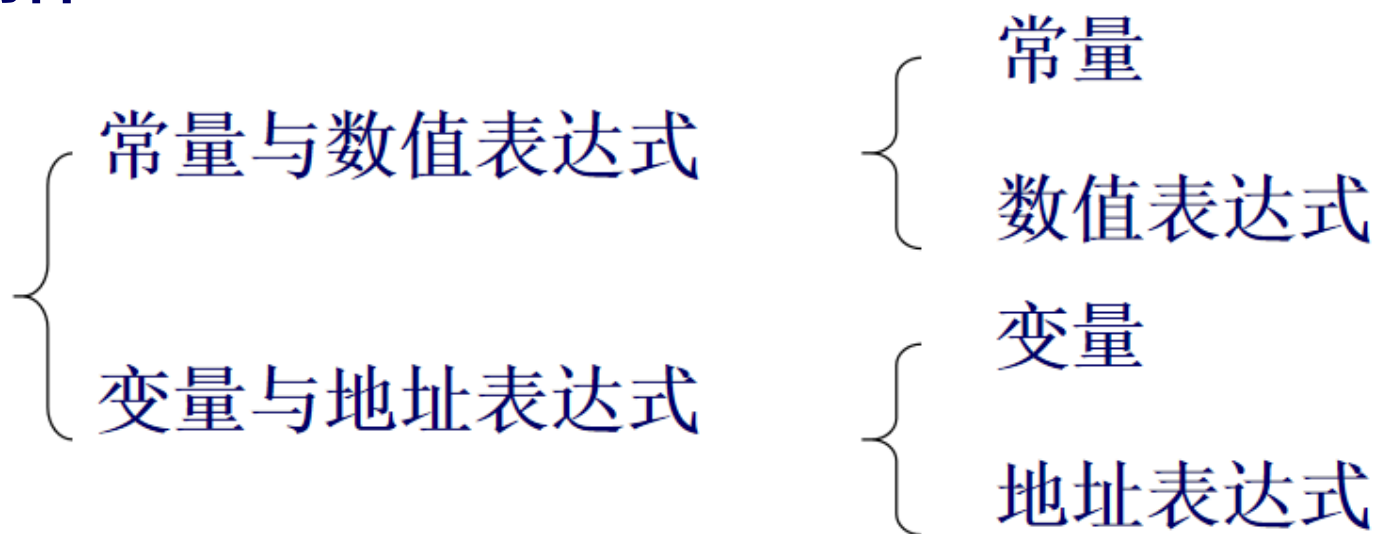
说明：

- ◆ 寄存器间接寻址，变址寻址和基址加变址寻址都能用来传送一片连续存储区的内容。
- ◆ a), b), c) 三种连续寻址中，b) 的寻址最直观，可读性最好。



5.常量和变量的定义

汇编程序的语句中具体的操作数，可以分为常量和变量两种：



表达式：由常数、寄存器、标号、变量加上运算符构成的式子





(1) 常量

常量：汇编时已有确定数值的量。

用途：赋值、作立即数、位移量。

常量的分类：{
数值常量：二进制、十进制、十六进制、字符
符号常量：有名字的数值常量

表达式：由常数、寄存器、标号、变量加上运算符构成的式子





符号常量的定义

◆ 等价伪指令 **.equ**

◆ 等号伪指令 **=**

使用：定义后直接引用符号名。

注意：

◆ 符号常量不分配存储单元，只建立等价代换关系，可出现在任何段。

◆ 用.equ语句定义的符号常量在该程序中不能再重新赋值，而用“=”定义的符号常量可多次重新赋值，使用时，以最后一次定义的值为准。





(2) 数值表达式

常量与运算符组成的式子。数值表达式在汇编期间进行运算，结果为常量。

◆ 算术运算

+、-、*、/、MOD(模除, 取余数)、
SHR(右移)、SHL(左移)。

◆ 逻辑运算

逻辑乘 AND (与), 逻辑加 OR (或), 按位加 XOR (异或), 逻辑非 NOT (非)





(2)数值表达式

◆ 关系运算

相等 EQ、不等 NE、小于 LT、大于 GT、小于等于 LE、
大于等于 GE

运算结果：关系不成立，结果为0；关系成立，结果为
-1 (0FFFFH) 。



(3) 变量



华中科技大学

数据段或附加数据段中一个数据存贮单元的名字, 是这个存储单元的地址的符号表示。可代表一批存储单元的首址。





1) 变量的属性

- 段属性：定义变量所在段;
- 偏移地址：该变量所占存储单元到所在段的段首址的字节距离;
- **类型**：类型是指存取该变量中的数据所需要的**字节数**, 变量的类型由定义该变量时所使用的伪指令确定;





2) 变量的定义

格式: [变量名:] 变量定义伪指令 表达式[, ...]

变量定义伪指令:

伪指令	数据类型
<code>.ascii</code>	文本字符串
<code>.asciiz/.string</code>	以0结尾的文本字符串
<code>.byte</code>	8位字节变量
<code>.word/.short/.hword</code>	16位字变量
<code>.long/.int</code>	32位双字变量
<code>.quad</code>	64位四字变量
<code>.octa</code>	128位八字变量
<code>.float/.single</code>	32位单精度浮点数
<code>.double</code>	64位双精度浮点数





2) 变量的定义

格式: [变量名:] 变量定义伪指令 表达式[, ...]

表达式:

- a) 数值表达式**
- b) ASCII字符串**
- c) 地址表达式**

上述(a) ~ (c)组成的系列, 各表达式之间用逗号隔开。



3) 示例

数据段定义如下:

```
# var_def.s
```

```
.section .data
```

```
msg:      .ascii "abc" , "123" , "\n\0"
```

```
      .equ con, 0x500
```

```
a1: .word 0xffaa
```

```
mark = 0x100
```

```
b1: .long 1122
```

```
buf:.byte 1, 2, 3, '1' , 'a'
```

```
d1: .long b1
```

请画出数据段中的数据在主存中的存储形式。





3) 示例

msg	0x61	0x0804a000	buf	0x00	0x0804a00c
	0x62	0x0804a001		0x00	0x0804a00d
	0x63	0x0804a002		0x01	0x0804a00e
	0x31	0x0804a003		0x02	0x0804a00f
	0x32	0x0804a004		0x03	0x0804a010
	0x33	0x0804a005		0x31	0x0804a011
	0x0a	0x0804a006		0x61	0x0804a012
a1	0x00	0x0804a007	d1	0x0a	0x0804a013
	0xaa	0x0804a008		0xa0	0x0804a014
	0xff	0x0804a009		0x04	0x0804a015
b1	0x62	0x0804a00a		0x08	0x0804a016
	0x04	0x0804a00b			

```
(gdb) x /200xb &msg
```

```
0x804a000:  0x61  0x62  0x63  0x31  0x32  0x33  0x0a  0x00
0x804a008:  0xaa  0xff  0x62  0x04  0x00  0x00  0x01  0x02
0x804a010:  0x03  0x31  0x61  0x0a  0xa0  0x04  0x08  Cannot
```





3) 示例

在上例中分别执行语句后AL的结果。

`movb buf, %al` ; %al = 0x01

`movw buf + 2, %ax` ; %ax = 0x3103

指令 `mov buf, %edx` 是否正确?

正确, 执行 `movl buf, %edx`

注意:

伪指令 EQU 及 “=” 不分配存贮单元;

使用直接寻址方式时, 变量的类型必须与指令的要求相符;





(4) 地址表达式

由变量、标号、常量、寄存器(名加括号)及一些运算符(数值表达式的运算符和特殊运算符)所组成的有意义的式子。

地址表达式也具有段、EA、类型等三个属性。

简单的地址表达式：直接寻址、间接寻址、变址、基址加变址等。





(4) 地址表达式

- 取偏移地址

\$地址表达式

功能：分离出变量、标号的偏移地址。

例：

movl \$buf, %esi

将变量buf的偏移地址，传送到esi中

\$buf是一个常量



(4) 地址表达式

- 当前即将分配的地址

功能：表示汇编程序当前即将分配的地址

格式：.

例：

```
.section .data
```

```
buf: .ascii "12345"
```

```
con      = . - buf
```

“.”表示当前即将分配的地址，“. - buf”表示当前地址减去buf的偏移地址，结果即为**变量buf的字节长度**

“. - buf”在汇编时计算，**结果为一个常数值**





(4) 地址表达式

- 填充伪指令

格式: **.fill repeat, size, value**

功能: 使用初始值, 重复repeat次填充size个字节

例:

.fill 10, 2, 0x5656

分配20个字节, 用word值0x5656, 填充10次。

小结



华中科技大学

◆ 主存物理地址的形成

◆ 寻址方式

◆ 常量和变量的定义

内存示意图



小结



华中科技大学

寻址方法

立即寻址方式

寄存器寻址方式

直接寻址方式

寄存器间接寻址方式

变址寻址方式

基址加变址寻址方式

